

金快查 — 全市场 LOF 基金实时折溢价监控系统 技术文档

一、项目概述

金快查是一个 LOF (Listed Open-end Fund, 上市型开放式基金) 全市场实时折溢价监控系统, 覆盖深沪两市全部 LOF 基金 (~540 只), 提供 Web 端与微信小程序双端访问, 帮助投资者发现套利机会。

生产地址

- 前端: <https://lof-fund-monitor.pages.dev>
- 后端: <https://lof-premium-tracker-production.up.railway.app>

二、前端架构

2.1 技术栈

层级	技术
部署	Cloudflare Pages + Functions
框架	原生 HTML5 / CSS3 / ES6 (零依赖 SPA)
图表	Chart.js 4.4 (CDN 加载, 仅详情弹窗按需使用)
样式	CSS 自定义属性 + 媒体查询响应式
网络	fetch() + 自定义重试逻辑

2.2 文件结构

```
/
├── index.html      # 单页入口, DOM 骨架
├── js/
│   ├── config.js  # API 地址、阈值等运行配置
│   ├── api.js     # 网络层: 请求封装、重试、健康检查
│   └── app.js      # 业务逻辑: 渲染、排序、筛选、弹窗、Chart
├── css/
│   └── style.css   # 全局样式 + 暗色模式 + 响应式
├── assets/
│   └── icon.jpg    # 品牌图标
├── pages/
│   ├── agreement.html # 用户协议
│   └── privacy.html   # 隐私政策
└── functions/      # Cloudflare Functions (代理层)
```

2.3 双端渲染策略

PC 端 (min-width: 769px)

- 左侧 280px 侧边栏: 溢价/折价 TOP5 + 图例

- - 右侧主内容区：表格，12 列（代码、名称、现价、净值、涨跌幅、溢价率、三日均溢、成交额、预计收益率、预计收益额、状态、净值日期）
- - 表头可点击排序，支持升序/降序切换
- - 底部可切换每页条数（50/100/200）

移动端（max-width: 768px）

- - 隐藏侧边栏与表格
- - 卡片列表 mobile-card-list：每张卡片显示代码、名称、溢价率、预计收益
- - 搜索栏下方显示溢价/折价模式切换按钮 + 套利流程帮助？
- - 设置按钮、分页控件紧凑排列

响应式断点

- - 600px：弹窗、字体、间距适配小屏手机
- - 769px：PC/移动布局切换

2.4 核心模块：`app.js`

类 LofFundMonitor，全局单例 lofMonitor，挂载 window。

功能	方法	说明
初始化	init()	健康检查 → 加载排行 → 加载全量基金
渲染表格	renderTable()	PC 端 行，含颜色标记与 ? 按钮
渲染卡片	renderMobileCards()	移动端卡片列表
排序	handleSort(field)	切换排序字段及方向
模式切换	setSortMode(mode)	移动端溢价/折价模式
搜索	handleSearch(kw)	代码或名称模糊匹配，300ms 防抖
分页	goToPage(n)	前端分页
预计收益	calcEstimatedProfit(fund)	含费率明细（申购/赎回/佣金）
收益弹窗	showProfitDetail(code)	套利收益明细弹窗
详情弹窗	showFundDetail(code)	12 项 KPI + Chart.js 双线图
Chart	_renderDetailChart(data)	场内价格 vs 场外净值折线图
费率明细	_toggleFeeBreakdown()	展开/关闭费用计算明细
暗色模式	toggleDarkMode()	CSS 变量切换 + localStorage
设置	openSettingsModal()	金额阈值、每份金额配置
刷新	handleManualRefresh()	元宝跳跃动画
Toast	showToast(msg)	临时提示条

2.5 事件委托

document.addEventListener('click') 统一处理以下交互，避免逐行绑定：

- - .fund-row / .mobile-card → 基金详情弹窗

- .btn-profit-info / .mc-profit-help → 收益构成弹窗
- #fdProfitHelp → 费率明细切换
- .fd-info-icon → 图表信息提示
- #fdCloseBtn / #fundDetailModal 背景 → 关闭详情
- .col-code / .col-name → 复制代码/名称（不触发详情）

2.6 网络层：`api.js`

方法	端点	说明
checkHealth()	GET /health	服务器状态 + 历史数据天数
getFunds(params)	GET /api/funds	分页 + 排序 + 搜索 + 筛选
getFundDetail(code)	GET /api/funds/{code}	单只基金数据（含申购状态）
getFundChart(code)	GET /api/funds/{code}/chart	7 交易日价格/净值曲线
requestWithRetry(url)	—	3 次重试，指数退避

2.7 暗色模式

- CSS 变量 var(--bg), var(--card), var(--text) 等全局控制
- 切换 .dark-mode 类
- localStorage 持久化：lof_dark_mode = '1'
- Chart.js 图例/坐标轴颜色随模式动态调整

三、后端架构

3.1 技术栈

层级	技术
部署	Railway（RAILPACK 自动构建）
框架	Flask 2.3 + Gunicorn
数据库	PostgreSQL（premium_snapshots 表）
数据源	东方财富 push2delay / push2his / fundgz / 天天基金
并发	threading（8 并发信号量，分批抓取）
缓存	内存 dict + threading.RLock 读写锁

3.2 文件结构

```
backend/  
├─ app.py      # Flask 路由：API、健康检查、手动刷新  
├─ config.py   # 配置常量（超时、间隔、费率阈值）  
├─ data_fetcher.py # 核心：多数据源抓取 + 溢价率计算  
├─ fee_fetcher.py # 费率爬虫：申购/赎回费率 + 转场内限制  
├─ history_fetcher.py # 历史数据：日 K 线 + 净值（过去 7 交易日）  
├─ history_db.py  # PostgreSQL 读写：快照存储、查询、清理
```

- └─ history_seed.json # 初始化种子数据
- └─ sz_lof_codes.json # 深市 LOF 代码缓存（每周自动刷新）
- └─ requirements.txt # Python 依赖

3.3 数据抓取架构

```
fetch_all() [每 5 分钟或手动触发]
├─
├─ Step 1: SSE LOF 价格
│   └─ push2delay.eastmoney.com (m:1+t:9, 分页)
│       过滤 501xxx / 502xxx 代码
│       → {code: {price, change_pct, volume, amount, name}}
│
├─ Step 2: SZ LOF 价格
│   └─ 代码列表: sz_lof_codes.json（每周自动刷新）
│       └─ push2delay (m:0+t:9) 扫描 16xxxx / 184xxx
│       行情: Tencent qt.gtimg.cn（批量 100 只/次）
│       → {code: {price, prev_close, change_pct, volume, amount}}
│
├─ Step 3: 合并去重（SSE 优先）
│
├─ Step 4: NAV 净值（批量 25 只/次）
│   └─ fundgz.1234567.com.cn（天天基金估值 API）
│       → {code: {nav, nav_date, is_formal_nav, name}}
│
├─ Step 5: 溢价率计算
│   premium_rate = (price - nav) / nav × 100
│   premium_status = 溢价 | 折价 | 平价
│
├─ Step 6: 费率数据（异步，不阻塞）
│   └─ fee_fetcher: 天天基金 HTML 解析
│       → {code: {purchase_fee_rate, redemption_fee_rate, ...}}
│
└─ Step 7: 快照存储
    └─ save_snapshot() → PostgreSQL premium_snapshots
```

3.4 API 端点

方法	路径	参数	说明
GET	/api/funds	page, page_size, sort, order, search	全量基金列表（分页+排序+筛选）
GET	/api/funds/	—	单只基金详情
GET	/api/funds//chart	—	近 7 交易日价格/净值曲线
GET	/health	—	服务状态、缓存数量、历史数据天数
POST	/refresh	—	手动触发全量刷新
POST	/init-history	days (1-21)	手动补填历史数据

3.5 PostgreSQL 数据库

表: premium_snapshots

列	类型	说明
id	SERIAL	自增主键
date	DATE	日期 YYYY-MM-DD
code	VARCHAR(6)	基金代码
premium_rate	DOUBLE PRECISION	溢价率 %
price	DOUBLE PRECISION	场内价格
nav	DOUBLE PRECISION	场外净值
amount	DOUBLE PRECISION	成交额
name	VARCHAR(100)	基金名称

- 联合唯一约束: (date, code)
- INSERT ... ON CONFLICT ... DO UPDATE 幂等写入
- 保留 21 天, 超期自动清理
- Railway 重启后从 PostgreSQL 加载最新日期数据恢复缓存
- psycopg2 ThreadedConnectionPool (2-10 连接)

3.6 懒更新机制

- _trigger_lazy_refresh(): 用户请求时检查上次刷新时间
- 若距上次刷新 > 300 秒 → 后台线程异步执行 fetch_all()
- 刷新期间返回已有缓存数据, 不阻塞请求
- 启动时优先从 history_db 加载快照数据 (降级方案)

3.7 深市 LOF 代码自动扫描

- sz_lof_codes.json 含 _meta.updated_at 时间戳
- 每次 fetch_all() 检查距上次扫描是否 > 7 天
- 是 → 调用 push2delay m:0+t:9 扫描全量 SZ LOF (16xxxx + 184xxx)
- 新代码合并写入 JSON, Railway 重启不丢失

四、CF Functions 代理层

Cloudflare Functions 位于 functions/ 目录, 作为前端与 Railway 之间的代理层:

浏览器 → Cloudflare Pages → CF Functions → Railway (Flask)
(静态文件) (API 代理) (数据抓取)

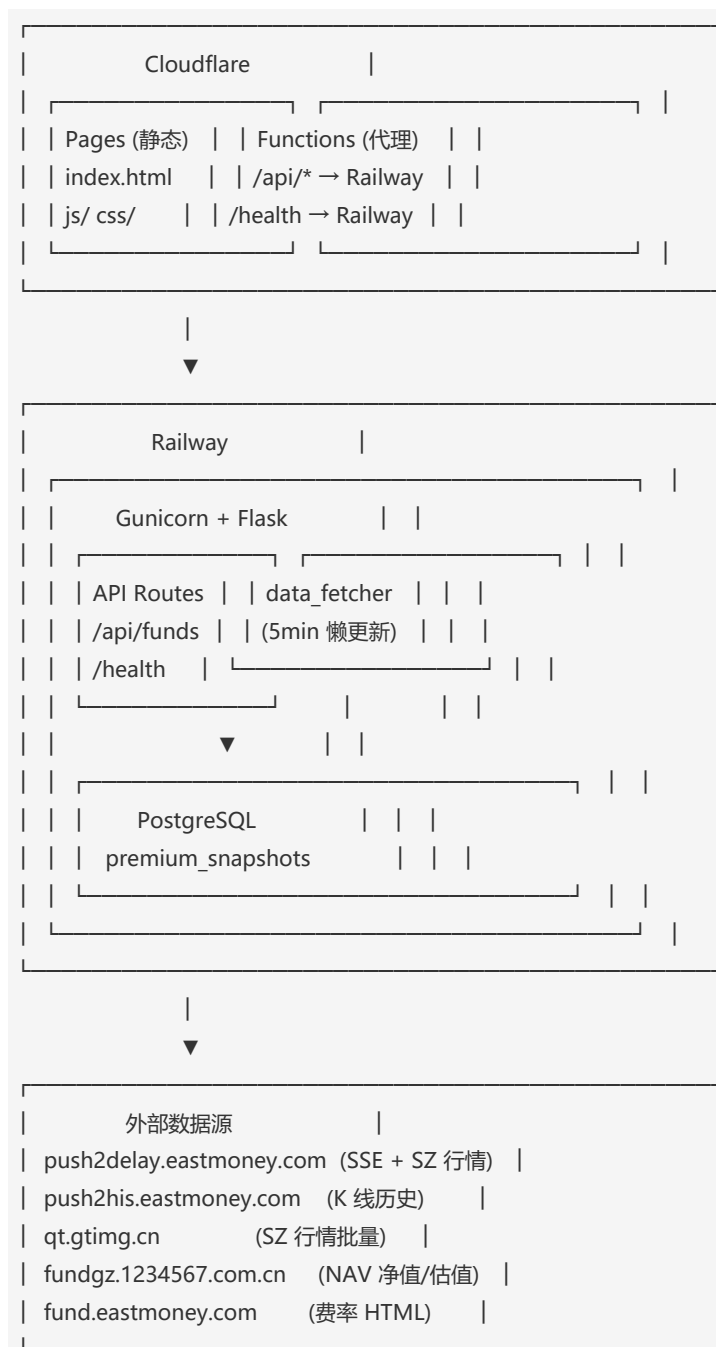
文件	路由	说明
functions/api/[[path]].js	/api/*	API 请求代理到 Railway, 添加 CORS 头
functions/health.js	/health	健康检查代理

为什么需要代理?

- Railway 后端 URL 直接暴露存在安全风险

- CF Functions 提供 CORS 处理
- 利用 CF 全球 CDN 加速 API 响应

五、部署架构



CI/CD

- 前端: git push main → GitHub Actions → Cloudflare Pages 自动部署
- 后端: git push main → Railway Webhook → RAILPACK 构建 → 自动部署
- 也可通过 npx wrangler pages deploy 手动部署前端

六、关键功能实现

6.1 溢价率计算

```
premium_rate = (场内价格 - 场外净值) / 场外净值 × 100
```

- 正值 = 溢价（场内价高于净值，申购套利机会）
- 负值 = 折价（场内价低于净值，赎回套利机会）
- 三日均溢：volume-weighted 3 日平均溢价率

6.2 预计收益计算（`calcEstimatedProfit`）

根据用户设置的金额阈值和每份金额：

- 溢价套利：收益 = 溢价率 - 申购费率 - 卖出佣金率
- 折价套利：收益 = 折价率 - 买入佣金率 - 赎回费率
- 考虑最低佣金收费（5 元起）
- 返回含费率明细的 breakdown 对象

6.3 基金详情弹窗

- 12 项 KPI（PC 全显示，移动端隐藏现价/净值/状态）
- 预计收益额概览条 + ? 按钮展开费率明细表格
- Chart.js 双线图：场内价格（橙）vs 场外净值（蓝）
- 图表数据来自 premium_snapshots 表（21 天窗口，覆盖约 7-10 个交易日）
- 切换基金时先 destroy() 旧 Chart 再创建新的
- 弹窗打开时锁定 body 滚动（overflow: hidden + overscroll-behavior: contain）

6.4 筛选与排序

功能	实现
溢价/折价筛选	<code>?filter=premium\</code>
停牌隐藏	默认隐藏 volume=0 的基金
不可申购隐藏	默认隐藏 can_purchase=false
搜索	前端实时过滤（代码+名称），300ms 防抖
排序	前端排序，支持 8 个字段
分页	前端分页 50/100/200 条/页

6.5 历史数据

- 每日懒更新自动保存当日快照到 PostgreSQL
- 手动触发 /init-history 补填过去 7 交易日数据
- K 线数据来自 push2his.eastmoney.com（日线 klt=101）
- 净值历史来自 api.fund.eastmoney.com/f10/lshz
- 保留 21 天，自动清理过期数据
- Railway 重启时从 PostgreSQL 恢复缓存

6.6 小程序

miniprogram/ 目录包含微信小程序源码：

- 首页：LOF 基金列表
- 详情页：基金详情 + 套利计算器
- 工具函数：utils/format.js（金额格式化）、utils/request.js（网络请求）

七、配置常量

`config.py`

常量	默认值	说明
REQUEST_TIMEOUT	10s	单次 API 请求超时
REFRESH_INTERVAL	300s	懒更新间隔
RETENTION_DAYS	21	历史数据保留天数
DEFAULT_AMOUNT_LIMIT	10000 元	默认金额阈值
DEFAULT_UNIT_AMOUNT	1000 元	默认每份金额

`config.js`

常量	说明
API_BASE_URL	后端 API 地址（可 URL 参数 ?api= 覆盖）
DEFAULT_PAGE_SIZE	默认每页条数 (50)

八、安全与性能

- 无鉴权：公开数据工具，无需登录
- CORS：CF Functions 统一添加 CORS 头
- 限流：CF Pages 自带 DDoS 防护；Railway 单 worker 限制并发
- 重试：前端 3 次指数退避重试；后端 3 次重试外部 API
- 并发控制：后端 8 线程信号量，分批 40 只/批，批间 0.8s 延迟
- 缓存：内存缓存 + PostgreSQL 持久化 + CDN 静态文件缓存
- 降级：外部 API 失败时使用历史快照数据